Project - Classification and Hypothesis Testing: Hotel Booking Cancellation Prediction

Marks: 40

Problem Statement

Context

A significant number of hotel bookings are called off due to cancellations or no-shows. Typical reasons for cancellations include change of plans, scheduling conflicts, etc. This is often made easier by the option to do so free of charge or preferably at a low cost. This may be beneficial to hotel guests, but it is a less desirable and possibly revenue-diminishing factor for hotels to deal with. Such losses are particularly high on last-minute cancellations.

The new technologies involving online booking channels have dramatically changed customers' booking possibilities and behavior. This adds a further dimension to the challenge of how hotels handle cancellations, which are no longer limited to traditional booking and guest characteristics.

This pattern of cancellations of bookings impacts a hotel on various fronts:

1. **Loss of resources (revenue)** when the hotel cannot resell the room.
2. **Additional costs of distribution channels** by increasing commissions or paying for publicity to help sell these rooms.
3. **Lowering prices last minute**, so the hotel can resell a room, resulting in reducing the profit margin.
4. **Human resources to make arrangements** for the guests.

Objective

This increasing number of cancellations calls for a Machine Learning based solution that can help in predicting which booking is likely to be canceled. INN Hotels Group has a chain of hotels in Portugal - they are facing problems with this high number of booking cancellations and have reached out to your firm for data-driven solutions. You, as a Data Scientist, have to analyze the data provided to find which factors have a high influence on booking cancellations, build a predictive model that can predict which booking is going to be canceled in advance, and help in formulating profitable policies for cancellations and refunds.

Data Description

The data contains the different attributes of customers' booking details. The detailed data dictionary is given below:

Data Dictionary

- **Booking_ID:** Unique identifier of each booking
- **no_of_adults:** Number of adults
- **no_of_children:** Number of children
- **no_of_weekend_nights:** Number of weekend nights (Saturday or Sunday) the guest stayed or booked to stay at the hotel
- **no_of_week_nights:** Number of weekday nights (Monday to Friday) the guest stayed or booked to stay at the hotel
- **type_of_meal_plan:** Type of meal plan booked by the customer:
  - Not Selected – No meal plan selected
  - Meal Plan 1 – Breakfast
  - Meal Plan 2 – Half board (breakfast and one other meal)
  - Meal Plan 3 – Full board (breakfast, lunch, and dinner)
- **required_car_parking_space:** Does the customer require a car parking space? (0 - No, 1- Yes)
- **room_type_reserved:** Type of room reserved by the customer. The values are ciphered (encoded) by INN Hotels.
- **lead_time:** Number of days between the date of booking and the arrival date
- **arrival_year:** Year of arrival date
- **arrival_month:** Month of arrival date
- **arrival_date:** Date of the month
- **market_segment_type:** Market segment designation.
- **repeated_guest:** Is the customer a repeated guest? (0 - No, 1- Yes)
- **no_of_previous_cancellations:** Number of previous bookings that were canceled by the customer prior to the current booking
- **no_of_previous_bookings_not_canceled:** Number of previous bookings not canceled by the customer prior to the current booking
- **avg_price_per_room:** Average price per day of the reservation; prices of the rooms are dynamic. (in euros)
- **no_of_special_requests:** Total number of special requests made by the customer (e.g. high floor, view from the room, etc)
- **booking_status:** Flag indicating if the booking was canceled or not.

Importing the libraries required

In [1]:
```
# Importing the basic libraries we will require for the project

# Libraries to help with reading and manipulating data
import pandas as pd
import numpy as np

# Libaries to help with data visualization
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

```
# Importing the Machine Learning models we require from Scikit-Learn
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
from sklearn.ensemble import RandomForestClassifier

# Importing the other functions we may require from Scikit-Learn
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import MinMaxScaler, LabelEncoder, OneHotEncoder

# To get diferent metric scores
from sklearn.metrics import confusion_matrix,classification_report,roc_auc_score,pl

# Code to ignore warnings from function usage
import warnings;
import numpy as np
warnings.filterwarnings('ignore')
```

Loading the dataset

In [2]:
```
hotel = pd.read_csv(r'C:\Users\andre\Downloads\INNHotelsGroup.csv')
```

In [3]:
```
# Copying data to another variable to avoid any changes to original data
data = hotel.copy()
```

Overview of the dataset

View the first and last 5 rows of the dataset

Let's **view the first few rows and last few rows** of the dataset in order to understand its structure a little better.

We will use the head() and tail() methods from Pandas to do this.

In [4]:
```
data.head()
```

Out[4]:

	Booking_ID	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights	type_of_me
0	INN00001	2	0	1	2	Mea
1	INN00002	2	0	2	3	Not S
2	INN00003	1	0	2	1	Mea
3	INN00004	2	0	0	2	Mea
4	INN00005	2	0	1	1	Not S

In [5]:
```
data.tail()
```

Out[5]:

	Booking_ID	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights	type_c
36270	INN36271	3	0	2	6	
36271	INN36272	2	0	1	3	
36272	INN36273	2	0	2	6	
36273	INN36274	2	0	0	3	
36274	INN36275	2	0	1	2	

Understand the shape of the dataset

In [6]:
```
data.shape
```

Out[6]:
```
(36275, 19)
```

- The dataset has 36275 rows and 19 columns.

Check the data types of the columns for the dataset

In [7]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 36275 entries, 0 to 36274
Data columns (total 19 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   Booking_ID                            36275 non-null  object
 1   no_of_adults                          36275 non-null  int64
 2   no_of_children                        36275 non-null  int64
 3   no_of_weekend_nights                  36275 non-null  int64
 4   no_of_week_nights                     36275 non-null  int64
 5   type_of_meal_plan                     36275 non-null  object
 6   required_car_parking_space            36275 non-null  int64
 7   room_type_reserved                    36275 non-null  object
 8   lead_time                             36275 non-null  int64
 9   arrival_year                          36275 non-null  int64
 10  arrival_month                         36275 non-null  int64
 11  arrival_date                          36275 non-null  int64
 12  market_segment_type                   36275 non-null  object
 13  repeated_guest                        36275 non-null  int64
 14  no_of_previous_cancellations          36275 non-null  int64
 15  no_of_previous_bookings_not_canceled  36275 non-null  int64
 16  avg_price_per_room                    36275 non-null  float64
 17  no_of_special_requests                36275 non-null  int64
 18  booking_status                        36275 non-null  object
dtypes: float64(1), int64(13), object(5)
memory usage: 5.3+ MB
```

- `Booking_ID`, `type_of_meal_plan`, `room_type_reserved`, `market_segment_type`, and `booking_status` are of object type while rest columns are numeric in nature.
- There are no null values in the dataset.

Dropping duplicate values

In [8]:
```
# checking for duplicate values
data.duplicated().sum()
```

Out[8]:
```
0
```

- There are **no duplicate values** in the data.

Dropping the unique values column

Let's drop the Booking_ID column first before we proceed forward, as a column with unique values will have almost no predictive power for the Machine Learning problem at hand.

In [9]:
```
data = data.drop(["Booking_ID"], axis=1)
```

In [10]:
```
data.head()
```

Out[10]:

	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights	type_of_meal_plan	requ
0	2	0	1	2	Meal Plan 1	
1	2	0	2	3	Not Selected	
2	1	0	2	1	Meal Plan 1	
3	2	0	0	2	Meal Plan 1	
4	2	0	1	1	Not Selected	

Question 1: Check the summary statistics of the dataset and write your observations (2 Marks)

Let's check the statistical summary of the data.

In [11]:
```
# Remove _________ and complete the code
data.describe().T
```

Out[11]:

	count	mean	std	min	25%	50%	
no_of_adults	36275.0	1.844962	0.518715	0.0	2.0	2.00	
no_of_children	36275.0	0.105279	0.402648	0.0	0.0	0.00	
no_of_weekend_nights	36275.0	0.810724	0.870644	0.0	0.0	1.00	
no_of_week_nights	36275.0	2.204300	1.410905	0.0	1.0	2.00	
required_car_parking_space	36275.0	0.030986	0.173281	0.0	0.0	0.00	
lead_time	36275.0	85.232557	85.930817	0.0	17.0	57.00	1
arrival_year	36275.0	2017.820427	0.383836	2017.0	2018.0	2018.00	20
arrival_month	36275.0	7.423653	3.069894	1.0	5.0	8.00	
arrival_date	36275.0	15.596995	8.740447	1.0	8.0	16.00	
repeated_guest	36275.0	0.025637	0.158053	0.0	0.0	0.00	
no_of_previous_cancellations	36275.0	0.023349	0.368331	0.0	0.0	0.00	
no_of_previous_bookings_not_canceled	36275.0	0.153411	1.754171	0.0	0.0	0.00	
avg_price_per_room	36275.0	103.423539	35.089424	0.0	80.3	99.45	1
no_of_special_requests	36275.0	0.619655	0.786236	0.0	0.0	0.00	

Write your answers here: - There are more adults than children visiting the hotel considering the mean, but surprisingly there is a higher value of maximum in the number of children, which maybe implies that the hotel very few times receives families. -According to the data, it is very uncommon that previous gests decide to come back again, which maybe implies that there are a lot of competitors or the competition is strong. Maybe the hotel doesn't provide an unique or excellent service. -There are more reservations in the week than in the weekends. Maybe that's because the hotel is focused on providing accommodation for business trips or to spend a short vacation. -Also, it seems that a lot of guests don't use the parking, so it is possible that the hotel gives them transportation.

Exploratory Data Analysis

Question 2: Univariate Analysis

Let's explore these variables in some more depth by observing their distributions.

We will first define a **hist_box() function** that provides both a boxplot and a histogram in the same visual, with which we can perform univariate analysis on the columns of this dataset.

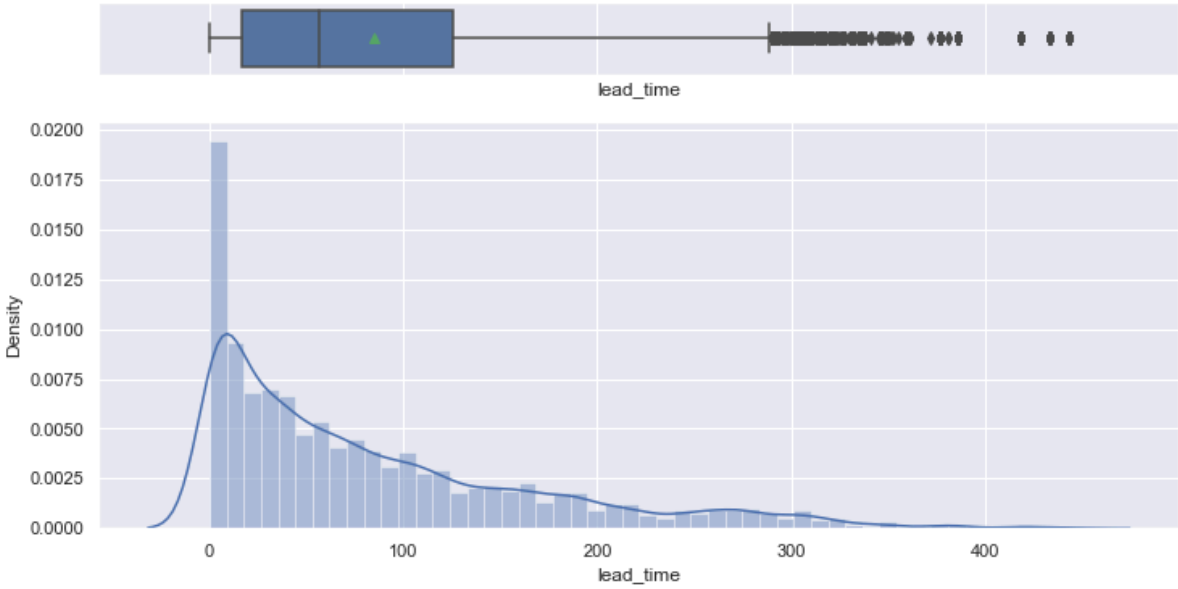
In [12]:

```
# Defining the hist_box() function
def hist_box(data,col):
  f, (ax_box, ax_hist) = plt.subplots(2, sharex=True, gridspec_kw={'height_ratios':
  # Adding a graph in each part
  sns.boxplot(data[col], ax=ax_box, showmeans=True)
```

```
sns.distplot(data[col], ax=ax_hist)
plt.show()
```

Question 2.1: Plot the histogram and box plot for the variable `Lead Time` using the hist_box function provided and write your insights. (1 Mark)

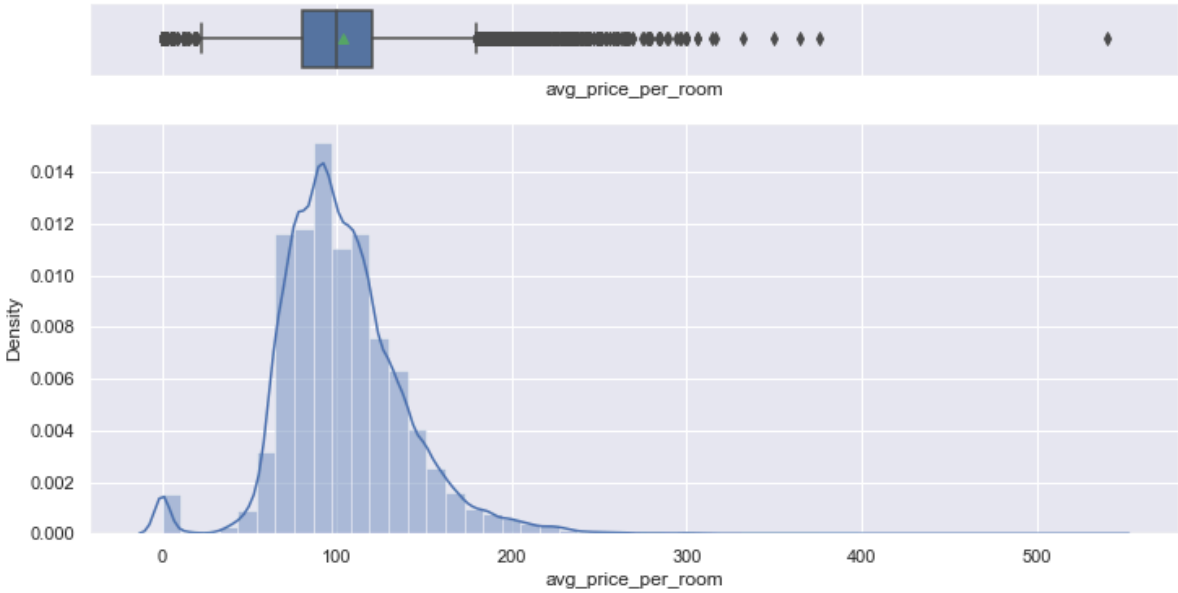
In [13]:
```
# Remove _________ and complete the code
hist_box(data, "lead_time")
```



Write your answers here: The majority of the guests prefer to book with a lot of anticipation, most of them book their reservations with 10-150 days of anticipation. Maybe we can implied that the guests do that because it is cheaper or they are used to visit their families on holidays and they want to secure a place. -Also, we find some outliers, so there is a significant number of guests that do their reservation with more than 150 days of anticipation, which is a lot of time and this can lead to more uncertainty.

Question 2.2: Plot the histogram and box plot for the variable `Average Price per Room` using the hist_box function provided and write your insights. (1 Mark)

In [14]:
```
# Remove _________ and complete the code
hist_box(data, "avg_price_per_room")
```



Write your answers here: We can see that the average price per room is about 100 euros, considering a width range of 70-170 euros. It is not so expensive, so maybe the guests are looking for an affordable accommodation with some comfort, but they are not eager to spend a lot of money. -Also, we can find a lot of outliers. Most of the outliers are not very far from each other, so there are a significant amount of people that are looking for a better service, but also they don't want to spend so much money. -Very few people want an expensive room.

Interestingly some rooms have a price equal to 0. Let's check them.

In [15]:
```
data[data["avg_price_per_room"] == 0]
```

Out[15]:

	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights	type_of_meal_plan
63	1	0	0	1	Meal Plan 1
145	1	0	0	2	Meal Plan 1
209	1	0	0	0	Meal Plan 1
266	1	0	0	2	Meal Plan 1
267	1	0	2	1	Meal Plan 1
...	...	...	...	...	...
35983	1	0	0	1	Meal Plan 1
36080	1	0	1	1	Meal Plan 1
36114	1	0	0	1	Meal Plan 1
36217	2	0	2	1	Meal Plan 1
36250	1	0	0	2	Meal Plan 2

545 rows × 18 columns

- There are quite a few hotel rooms which have a price equal to 0.
- In the market segment column, it looks like many values are complementary.

In [16]:
```
data.loc[data["avg_price_per_room"] == 0, "market_segment_type"].value_counts()
```

Out[16]:
```
Complementary    354
Online           191
Name: market_segment_type, dtype: int64
```

- It makes sense that most values with room prices equal to 0 are the rooms given as complimentary service from the hotel.
- The rooms booked online must be a part of some promotional campaign done by the hotel.

In [17]:
```
# Calculating the 25th quantile
Q1 = data["avg_price_per_room"].quantile(0.25)

# Calculating the 75th quantile
Q3 = data["avg_price_per_room"].quantile(0.75)

# Calculating IQR
IQR = Q3 - Q1

# Calculating value of upper whisker
Upper_Whisker = Q3 + 1.5 * IQR
Upper_Whisker
```

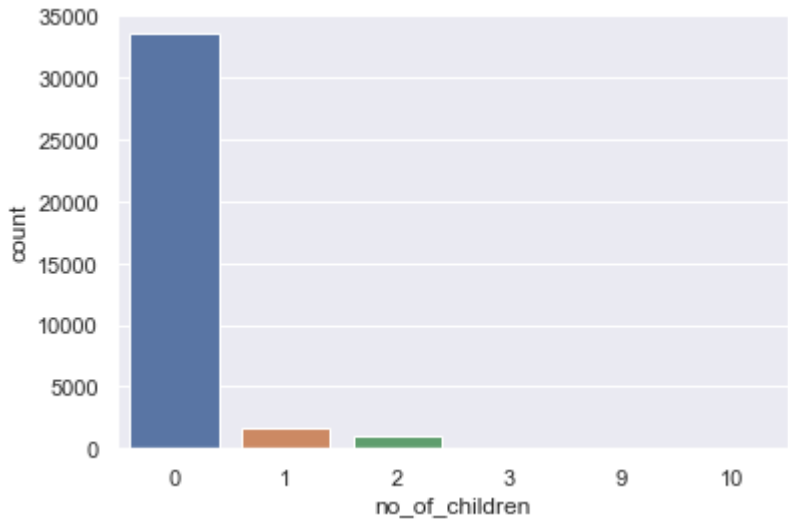
Out[17]:
```
179.55
```

In [18]:
```
# assigning the outliers the value of upper whisker
data.loc[data["avg_price_per_room"] >= 500, "avg_price_per_room"] = Upper_Whisker
```

Let's understand the distribution of the categorical variables

Number of Children

In [19]:
```
sns.countplot(data['no_of_children'])
plt.show()
```



In [20]:
```
data['no_of_children'].value_counts(normalize=True)
```

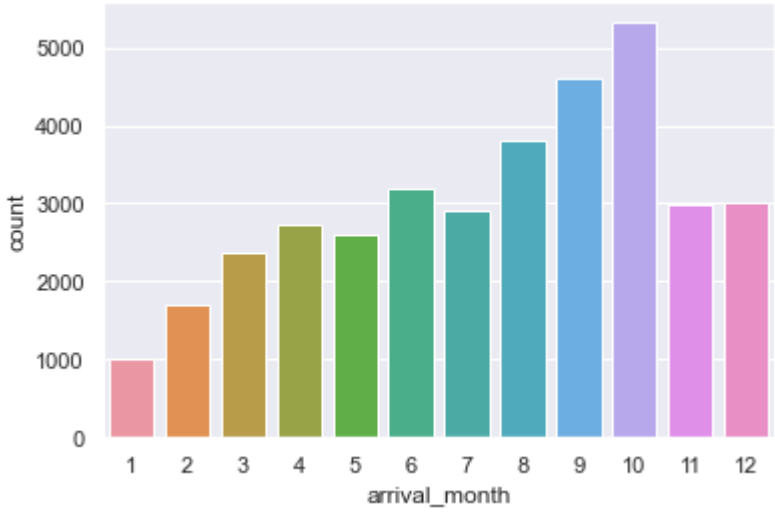
Out[20]:
```
0     0.925624
1     0.044604
2     0.029166
3     0.000524
9     0.000055
10    0.000028
Name: no_of_children, dtype: float64
```

- Customers were not travelling with children in 93% of cases.
- There are some values in the data where the number of children is 9 or 10, which is highly unlikely.
- We will replace these values with the maximum value of 3 children.

In [21]:
```
# replacing 9, and 10 children with 3
data["no_of_children"] = data["no_of_children"].replace([9, 10], 3)
```

Arrival Month

In [22]:
```
sns.countplot(data["arrival_month"])
plt.show()
```



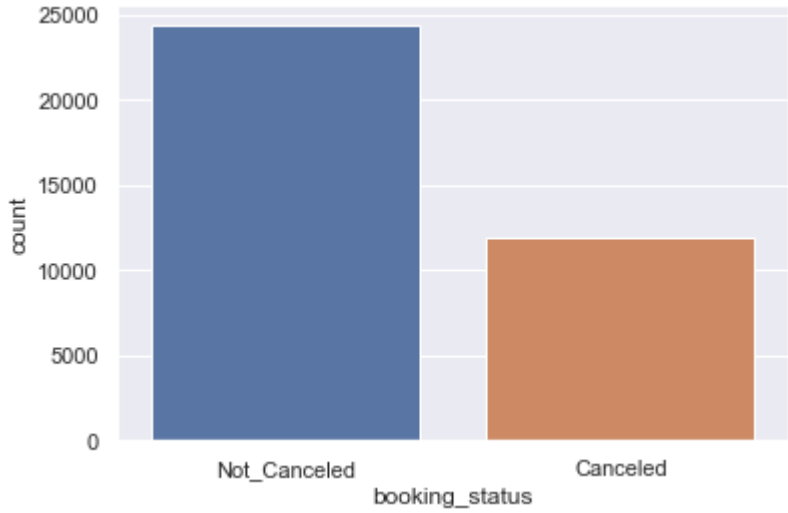
In [23]:
```
data['arrival_month'].value_counts(normalize=True)
```

Out[23]:
```
10    0.146575
9     0.127112
8     0.105114
6     0.088298
12    0.083280
11    0.082150
7     0.080496
4     0.075424
5     0.071620
3     0.065003
2     0.046975
1     0.027953
Name: arrival_month, dtype: float64
```

- October is the busiest month for hotel arrivals followed by September and August. **Over 35% of all bookings**, as we see in the above table, were for one of these three months.
- Around 14.7% of the bookings were made for an October arrival.

Booking Status

In [24]:
```
sns.countplot(data["booking_status"])
plt.show()
```



In [25]:
```
data['booking_status'].value_counts(normalize=True)
```

Out[25]:
```
Not_Canceled    0.672364
Canceled        0.327636
Name: booking_status, dtype: float64
```

- 32.8% of the bookings were canceled by the customers.

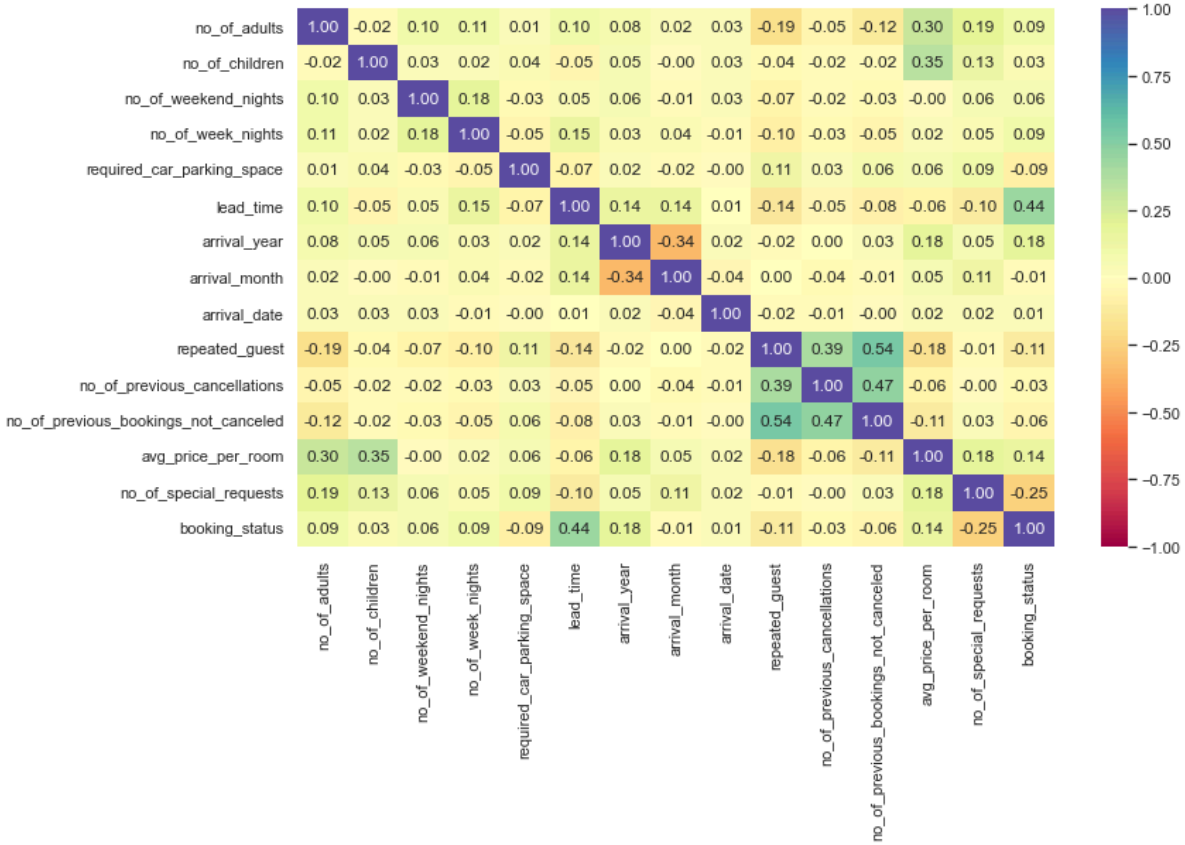
Let's encode Canceled bookings to 1 and Not_Canceled as 0 for further analysis

In [26]:
```
data["booking_status"] = data["booking_status"].apply(
    lambda x: 1 if x == "Canceled" else 0
)
```

Question 3: Bivariate Analysis

Question 3.1: Find and visualize the correlation matrix using a heatmap and write your observations from the plot. (2 Marks)

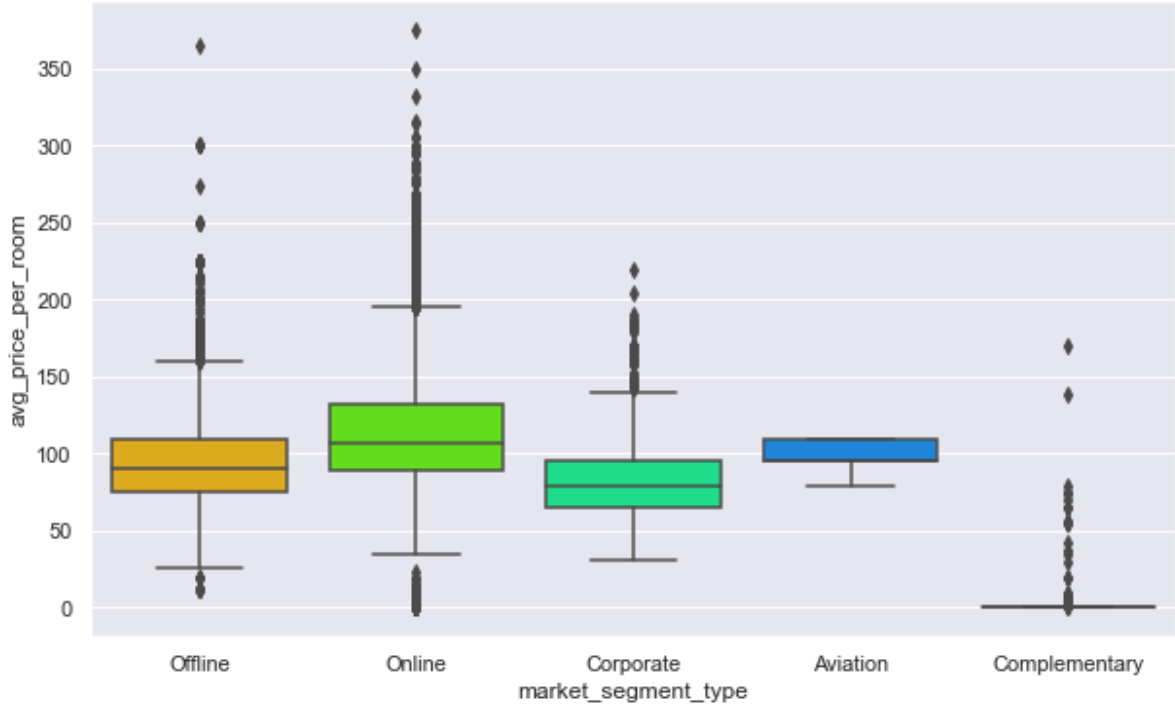
In [27]:
```
# Remove _________ and complete the code
cols_list = data.select_dtypes(include=np.number).columns.tolist()

plt.figure(figsize=(12, 7))
sns.heatmap(data[cols_list].corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="S
plt.show()
```



Write your answers here: There is a mild correlation (0.54) between repeated guests and the number of previous bookings not cancelled. -There is a mild correlation (0.47) between the number of previous bookings not cancelled and the number of previous cancellations. -There is a negative and pretty low correlation (-0.02) between the number of adults and the number of children, which implies that maybe there are not a lot of families visiting the hotel. -There is a shockingly low correlation (0.01) between the number of adults and the required car parking space, so I think that maybe a lot of them arrive to the hotel by special transportation, like a van or a bus for tourists. -There is no correlation (0.0) between the arrival year and the number of previous cancellations. That is similar to the arrival date, the arrival month and the lead time.

Hotel rates are dynamic and change according to demand and customer demographics. Let's see how prices vary across different market segments

In [28]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(
    data=data, x="market_segment_type", y="avg_price_per_room", palette="gist_rainb
)
plt.show()
```



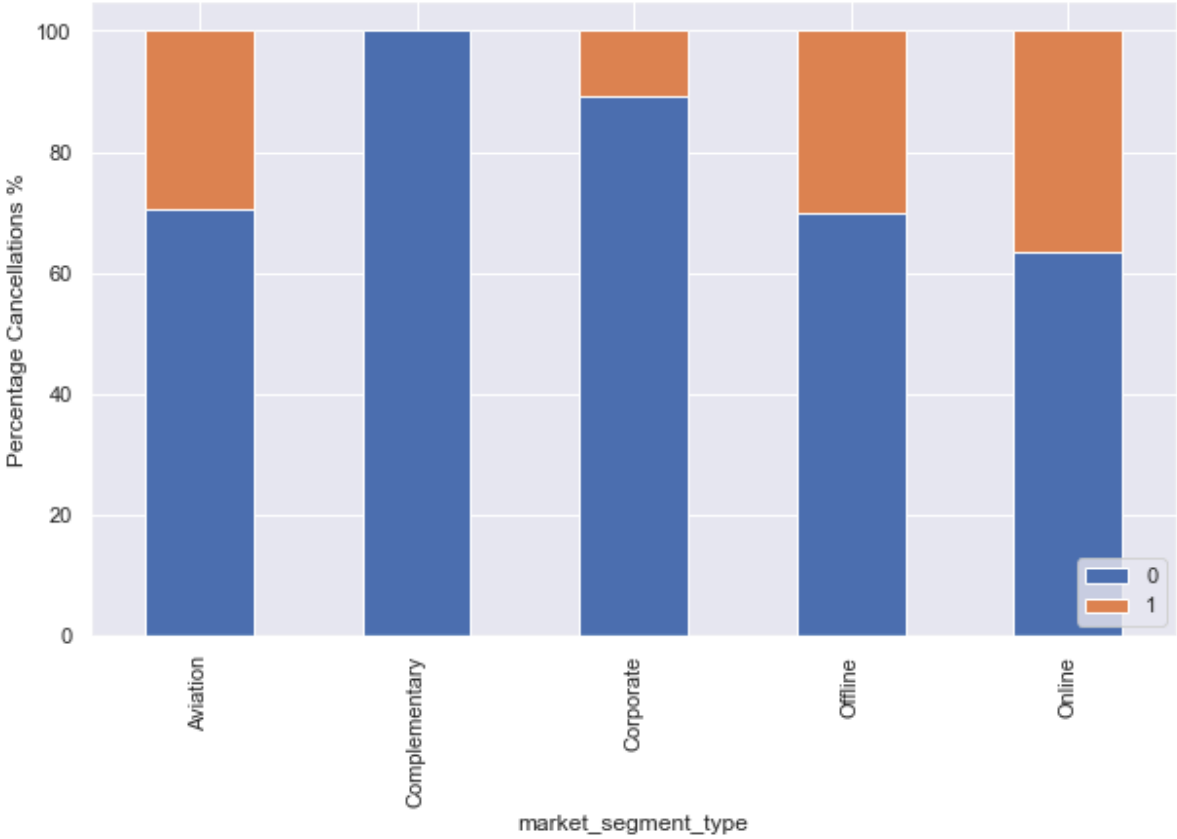
- Rooms booked online have high variations in prices.
- The offline and corporate room prices are almost similar.
- Complementary market segment gets the rooms at very low prices, which makes sense.

We will define a **stacked barplot()** function to help analyse how the target variable varies across predictor categories.

In [29]:
```
# Defining the stacked_barplot() function
def stacked_barplot(data,predictor,target,figsize=(10,6)):
  (pd.crosstab(data[predictor],data[target],normalize='index')*100).plot(kind='bar'
  plt.legend(loc="lower right")
  plt.ylabel('Percentage Cancellations %')
```

Question 3.2: Plot the stacked barplot for the variable `Market Segment Type` against the target variable `Booking Status` using the stacked_barplot function provided and write your insights. (1 Mark)

In [30]:
```
# Remove _________ and complete the code
stacked_barplot(data, "market_segment_type", "booking_status")
```

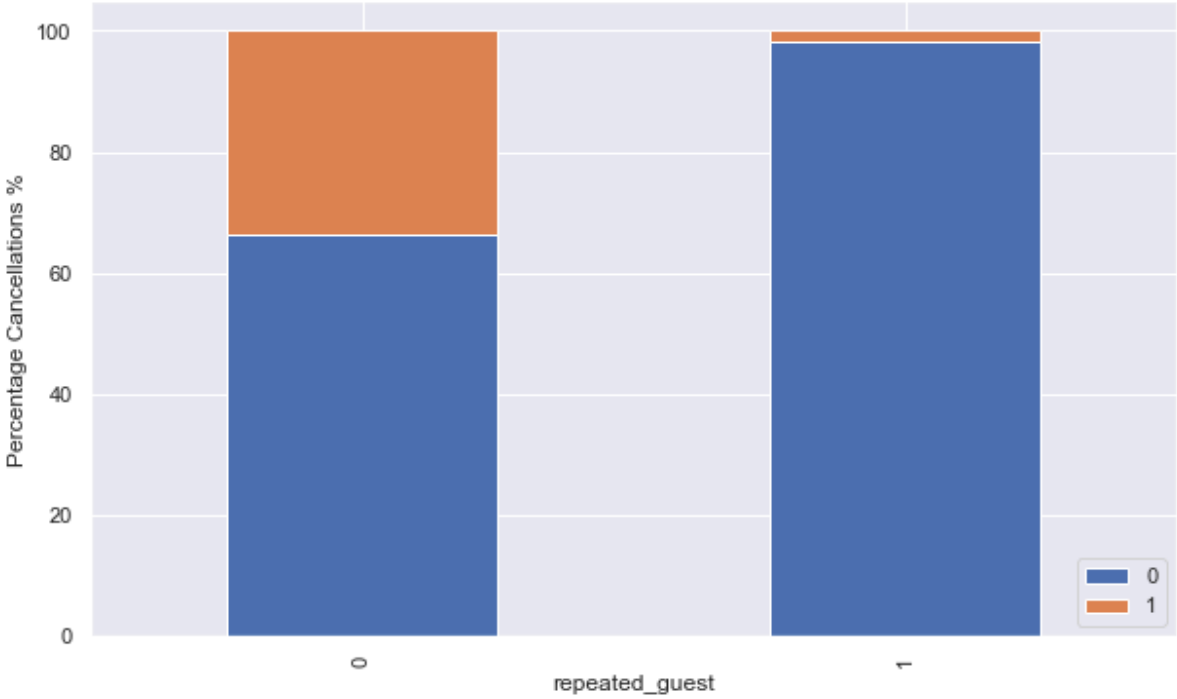


Write your answers here: In the online market segment type there is a higher percentage of cancellation than in the rest of the segments. It is not very surprising if we consider that some people make their reservation online only to secure a place but if they find something better, they change of opinion and it is easier to cancel online. -In the aviation and offline segment, they have very similar percentage of cancellation. -In the corporate segment, there is a little percentage of cancellation. Maybe it is because of business trips that are not frequently affected by cancellations. -In the complementary segment, we found no percentage of cancellation, which is surprising.

Question 3.3: Plot the stacked barplot for the variable `Repeated Guest` against the target variable `Booking Status` using the stacked_barplot function provided and write your insights. (1 Mark)

Repeating guests are the guests who stay in the hotel often and are important to brand equity.

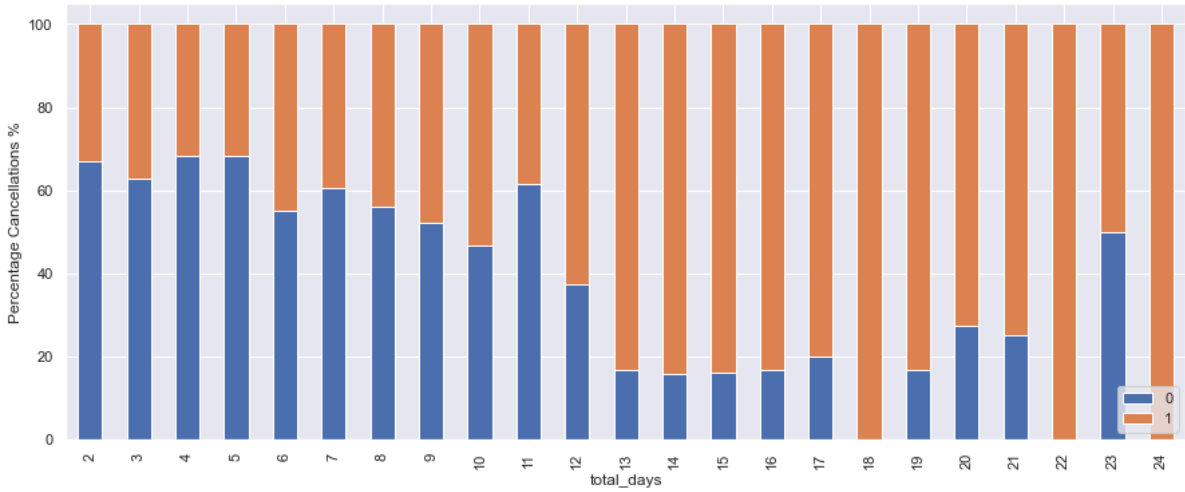
In [31]:
```
# Remove _________ and complete the code
stacked_barplot(data, "repeated_guest", "booking_status")
```



Write your answers here: - There is a higher percentage of cancellation when there are no repeated guests. Maybe that's because there are more non repeated guests. Also it could be because they have little knowledge about the hotel and this makes it easier to cancel. -In the case of the repeated guests, we find a much lower percentage of cancellation. Maybe that's because there are very few repeated guests and so on, the percentage of cancellation is smaller.

Let's analyze the customer who stayed for at least a day at the hotel.

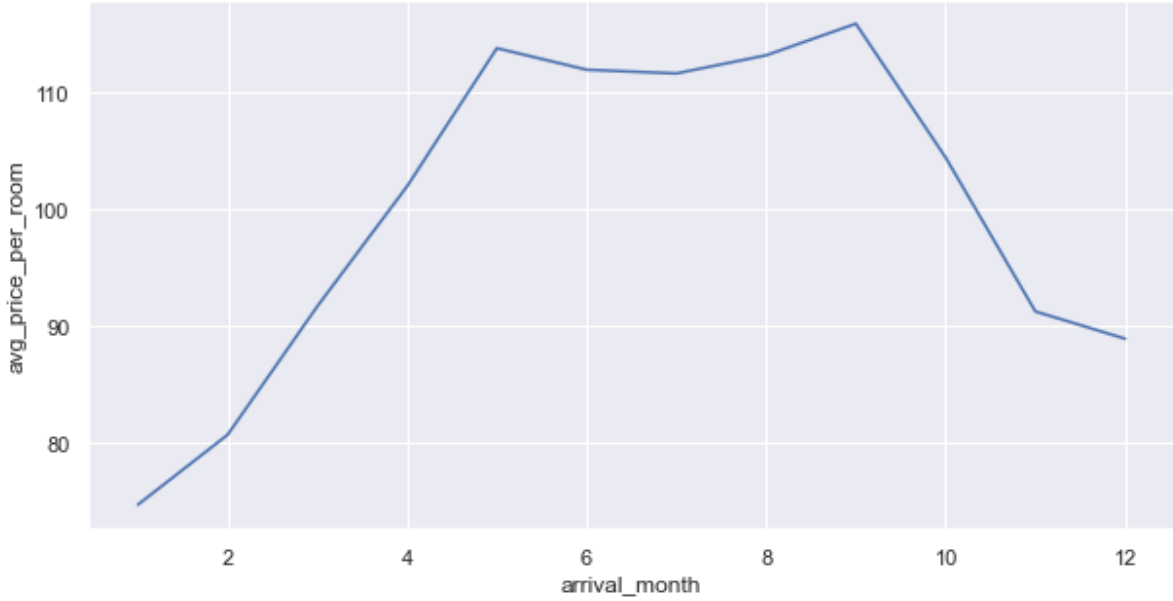
In [32]:
```
stay_data = data[(data["no_of_week_nights"] > 0) & (data["no_of_weekend_nights"] >
stay_data["total_days"] = (stay_data["no_of_week_nights"] + stay_data["no_of_weeker

stacked_barplot(stay_data, "total_days", "booking_status",figsize=(15,6))
```



- The general trend is that the chances of cancellation increase as the number of days the customer planned to stay at the hotel increases.

As hotel room prices are dynamic, Let's see how the prices vary across different months

In [33]:
```
plt.figure(figsize=(10, 5))
sns.lineplot(y=data["avg_price_per_room"], x=data["arrival_month"], ci=None)
plt.show()
```



- The price of rooms is highest in May to September - around 115 euros per room.

Data Preparation for Modeling

- We want to predict which bookings will be canceled.
- Before we proceed to build a model, we'll have to encode categorical features.
- We'll split the data into train and test to be able to evaluate the model that we build on the train data.

Separating the independent variables (X) and the dependent variable (Y)

In [34]:
```
X = data.drop(["booking_status"], axis=1)
Y = data["booking_status"]

X = pd.get_dummies(X, drop_first=True) # Encoding the Categorical features
```

Splitting the data into a 70% train and 30% test set

Some classification problems can exhibit a large imbalance in the distribution of the target classes: for instance there could be several times more negative samples than positive samples. In such cases it is recommended to use the **stratified sampling** technique to ensure that relative class frequencies are approximately preserved in each train and validation fold.

In [35]:
```
# Splitting data in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.30,stratify=Y
```

In [36]:
```
print("Shape of Training set : ", X_train.shape)
print("Shape of test set : ", X_test.shape)
print("Percentage of classes in training set:")
print(y_train.value_counts(normalize=True))
```

```
print("Percentage of classes in test set:")
print(y_test.value_counts(normalize=True))
```

```
Shape of Training set :  (25392, 27)
Shape of test set :  (10883, 27)
Percentage of classes in training set:
0    0.672377
1    0.327623
Name: booking_status, dtype: float64
Percentage of classes in test set:
0    0.672333
1    0.327667
Name: booking_status, dtype: float64
```

Model Evaluation Criterion

Model can make wrong predictions as:

1. Predicting a customer will not cancel their booking but in reality, the customer will cancel their booking.
2. Predicting a customer will cancel their booking but in reality, the customer will not cancel their booking.

Which case is more important?

Both the cases are important as:

- If we predict that a booking will not be canceled and the booking gets canceled then the hotel will lose resources and will have to bear additional costs of distribution channels.

- If we predict that a booking will get canceled and the booking doesn't get canceled the hotel might not be able to provide satisfactory services to the customer by assuming that this booking will be canceled. This might damage brand equity.

How to reduce the losses?

- The hotel would want the `F1 Score` to be maximized, the greater the F1 score, the higher the chances of minimizing False Negatives and False Positives.

Also, let's create a function to calculate and print the classification report and confusion matrix so that we don't have to rewrite the same code repeatedly for each model.

In [37]:
```
# Creating metric function
def metrics_score(actual, predicted):
    print(classification_report(actual, predicted))

    cm = confusion_matrix(actual, predicted)
    plt.figure(figsize=(8,5))

    sns.heatmap(cm, annot=True,  fmt='.2f', xticklabels=['Not Cancelled', 'Cancelle
    plt.ylabel('Actual')
    plt.xlabel('Predicted')
    plt.show()
```

Building the model

We will be building 4 different models:

- **Logistic Regression**
- **Support Vector Machine (SVM)**
- **Decision Tree**
- **Random Forest**

Question 4: Logistic Regression (6 Marks)

Question 4.1: Build a Logistic Regression model (Use the sklearn library) (1 Mark)

In [38]:
```
# Remove _________ and complete the code

# Fitting logistic regression model
lg = LogisticRegression()
lg.fit(X_train,y_train)
```

Out[38]:
```
▾ LogisticRegression
LogisticRegression()
```

Question 4.2: Check the performance of the model on train and test data (2 Marks)

In [39]:
```
# Remove _________ and complete the code

# Checking the performance on the training data
y_pred_train = lg.predict(X_train)
metrics_score(y_train, y_pred_train)
```

```
              precision    recall  f1-score   support

           0       0.82      0.89      0.85     17073
           1       0.73      0.60      0.66      8319

    accuracy                           0.80     25392
   macro avg       0.77      0.75      0.76     25392
weighted avg       0.79      0.80      0.79     25392
```

Write your Answer here: We have been able to build a predictive model that can be used by the hotel company to predict the guests who are likely to not cancel the reservation with a f-1 score of 85% We have been able to build a predictive model that can be used by the tourist company to predict the customers who are likely to cancel the reservation with a f-1 score of 66%.

Let's check the performance on the test set

In [40]:
```
# Remove _________ and complete the code

# Checking the performance on the test dataset
y_pred_test = lg.predict(X_test)
metrics_score(y_test, y_pred_test)
```

```
              precision    recall  f1-score   support

           0       0.82      0.89      0.85      7317
           1       0.72      0.59      0.65      3566

    accuracy                           0.79     10883
   macro avg       0.77      0.74      0.75     10883
weighted avg       0.79      0.79      0.79     10883
```

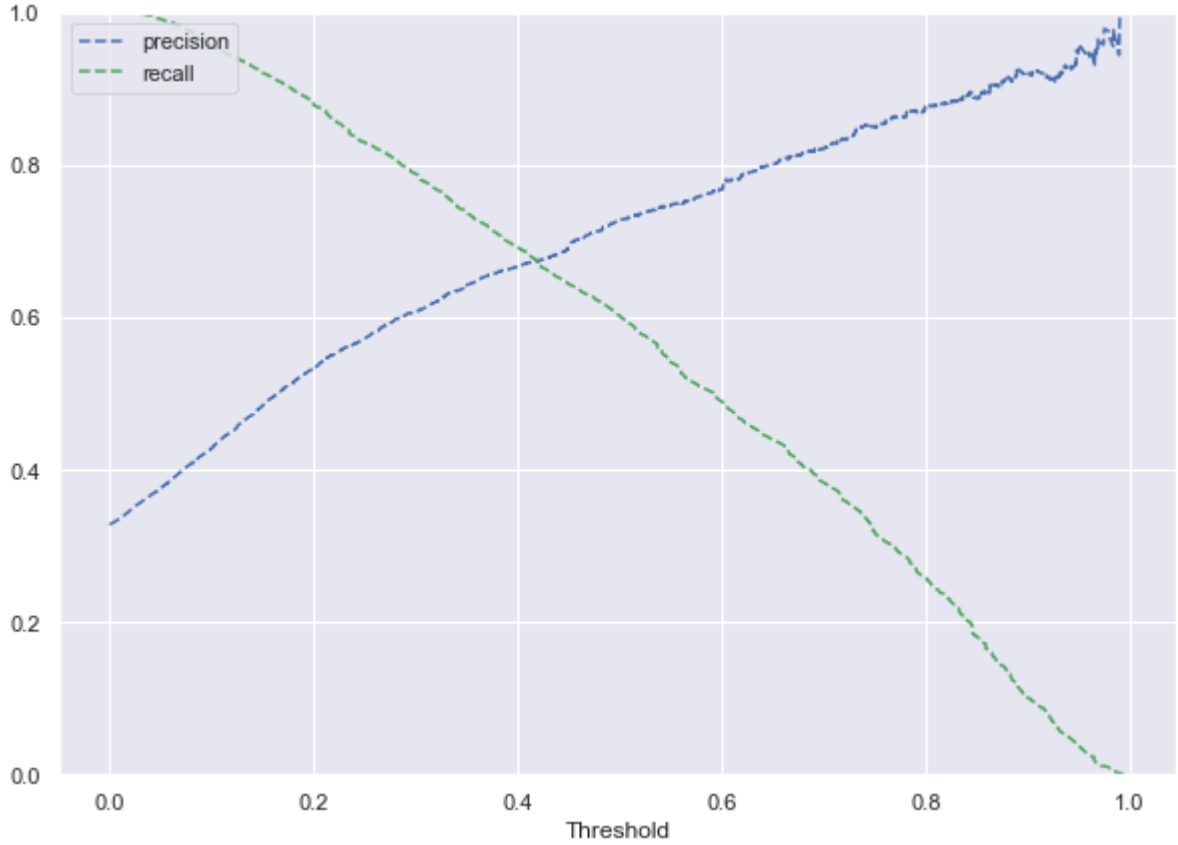
Write your Answer here: Considering the not canceled reservations, we have a high precision, a high recall, and a high f-1 score. Considering the canceled reservations, we have a good but lower precision, a mild recall and a decent f-1 score. We need to find the optimal threshold for the model.

Question 4.3: Find the optimal threshold for the model using the Precision-Recall Curve. (1 Mark)

Precision-Recall curves summarize the trade-off between the true positive rate and the positive predictive value for a predictive model using different probability thresholds.

Let's use the Precision-Recall curve and see if we can find a **better threshold.**

In [41]:
```
# Remove _________ and complete the code

# Predict_proba gives the probability of each observation belonging to each class
y_scores_lg=lg.predict_proba(X_train)

precisions_lg, recalls_lg, thresholds_lg = precision_recall_curve(y_train, y_scores

# Plot values of precisions, recalls, and thresholds
plt.figure(figsize=(10,7))
plt.plot(thresholds_lg, precisions_lg[:-1], 'b--', label='precision')
plt.plot(thresholds_lg, recalls_lg[:-1], 'g--', label = 'recall')
plt.xlabel('Threshold')
plt.legend(loc='upper left')
plt.ylim([0,1])
plt.show()
```



Write your answers here: We need the highest possible f-1 score. In this case, we have the precision and recall curve, so we need to select the threshold that obtains the highest precision and recall because the f-1 score becomes high only when both precision and recall are high. So the threshold value of 0.4 should be sufficient because it has a good recall and precision.

In [42]:
```
# Setting the optimal threshold
optimal_threshold = 0.4
```

Question 4.4: Check the performance of the model on train and test data using the optimal threshold. (2 Marks)

In [43]:
```
# Remove _________ and complete the code

# Creating confusion matrix
y_pred_train = lg.predict_proba(X_train)
metrics_score(y_train, y_pred_train[:,1]>optimal_threshold)
```

```
              precision    recall  f1-score   support

           0       0.85      0.83      0.84     17073
           1       0.67      0.69      0.68      8319

    accuracy                           0.79     25392
   macro avg       0.76      0.76      0.76     25392
weighted avg       0.79      0.79      0.79     25392
```

Write your answers here: The model performance has improved as compared to our initial model. The f-1 score of the canceled reservations has increased 3%.

Let's check the performance on the test set

In [44]:
```
# Remove _________ and complete the code

y_pred_test = lg.predict_proba(X_test)
metrics_score(y_test, y_pred_test[:,1]>optimal_threshold)
```

```
              precision    recall  f1-score   support

           0       0.84      0.83      0.84      7317
           1       0.66      0.69      0.67      3566

    accuracy                           0.78     10883
   macro avg       0.75      0.76      0.75     10883
weighted avg       0.78      0.78      0.78     10883
```

Write your answers here: Using the model with a threshold of 0.4, the model has achieved a f-1 score of 67%. The f-1 score hasn't dropped so much compared to inital

model but using an optimal threshold the model is able to provide a balanced score.

Question 5: Support Vector Machines (11 Marks)

To accelerate SVM training, let's scale the data for support vector machines.

In [45]:
```
scaling = MinMaxScaler(feature_range=(-1,1)).fit(X_train)
X_train_scaled = scaling.transform(X_train)
X_test_scaled = scaling.transform(X_test)
```

Let's build the models using the two of the widely used kernel functions:

1. **Linear Kernel**
2. **RBF Kernel**

Question 5.1: Build a Support Vector Machine model using a linear kernel (1 Mark)

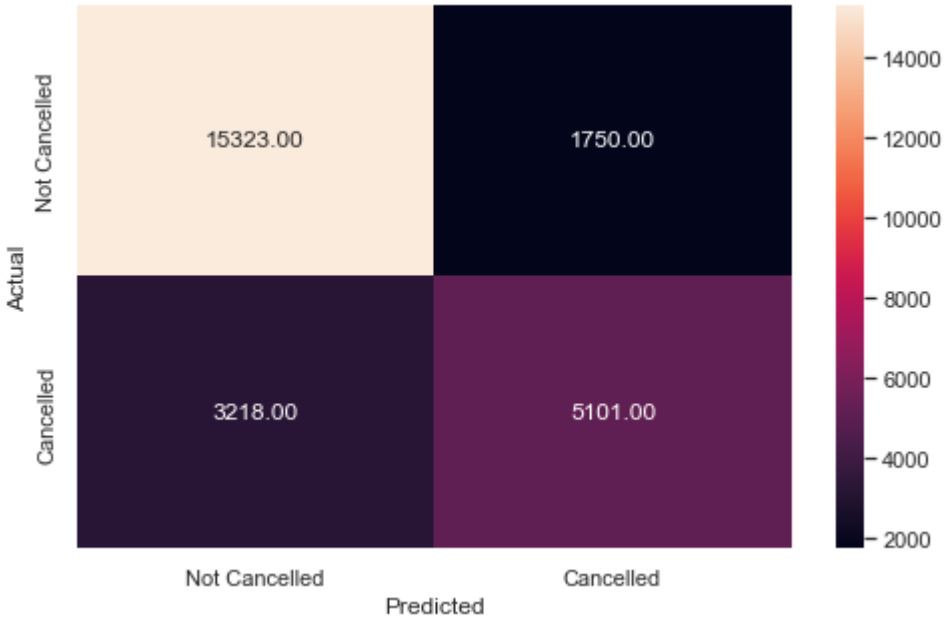
Note: Please use the scaled data for modeling Support Vector Machine

In [46]:
```
# Remove _________ and complete the code

svm = SVC(kernel='linear',probability=True) # Linear kernal or linear decision bour
model = svm.fit(X= X_train_scaled, y = y_train)
```

Question 5.2: Check the performance of the model on train and test data (2 Marks)

In [47]:
```
# Remove _________ and complete the code

y_pred_train_svm = model.predict(X_train_scaled)
metrics_score(y_train, y_pred_train_svm)
```

```
              precision    recall  f1-score   support

           0       0.83      0.90      0.86     17073
           1       0.74      0.61      0.67      8319

    accuracy                           0.80     25392
   macro avg       0.79      0.76      0.77     25392
weighted avg       0.80      0.80      0.80     25392
```



Write your answers here: The f-1 score is good, we have a 86% in not cancelled reservations and 67% in cancelled reservations. -But the recall in the cancelled reservations is somewhat low, so it is important to figure out if another model can provide us a better value.

Checking model performance on test set

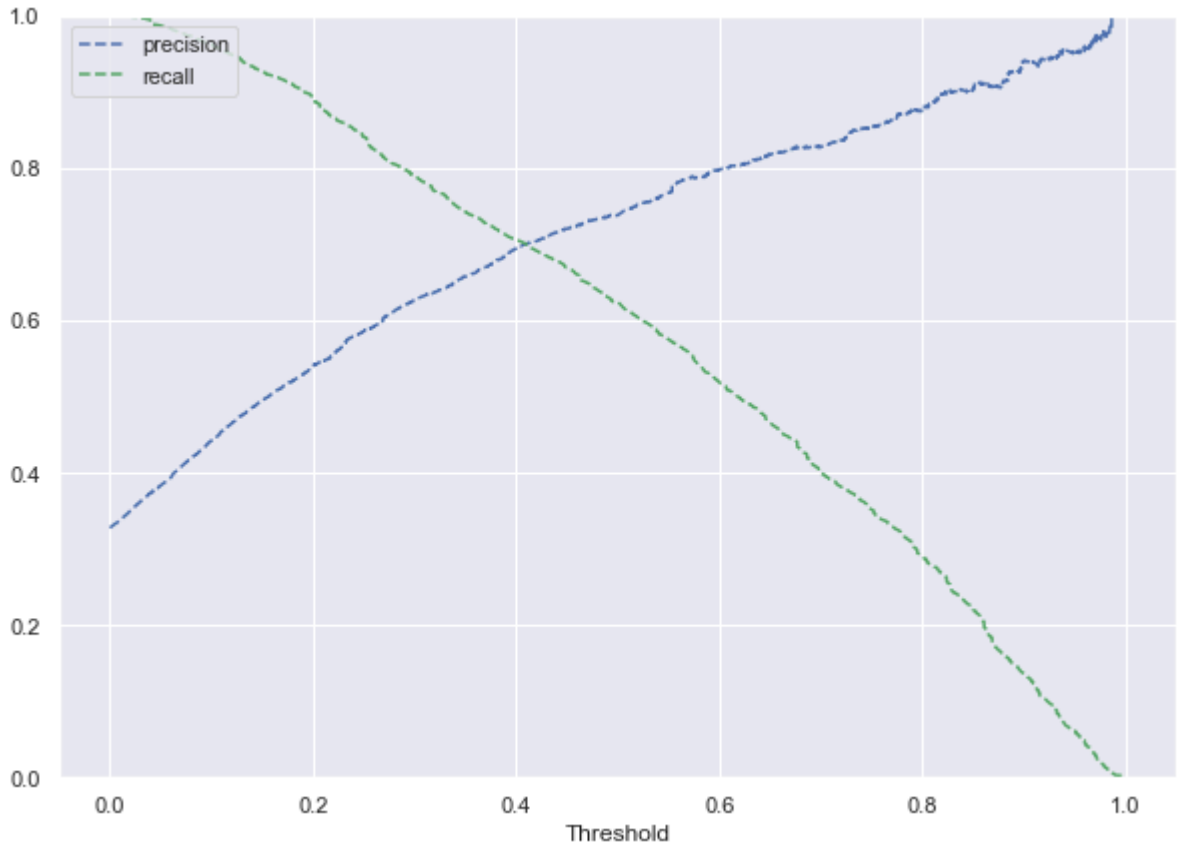
In [48]:
```
# Remove _________ and complete the code

y_pred_test_svm = model.predict(X_test_scaled)
metrics_score(y_test, y_pred_test_svm)
```

```
              precision    recall  f1-score   support

           0       0.82      0.90      0.86      7317
           1       0.74      0.61      0.67      3566

    accuracy                           0.80     10883
   macro avg       0.78      0.75      0.76     10883
weighted avg       0.80      0.80      0.80     10883
```

Write your answers here: The f-1 score is good, the same as in the training model. -The recall and precision values are basically the same, and the precision value of the not cancelled reservations is pretty similar to the training model.

Question 5.3: Find the optimal threshold for the model using the Precision-Recall Curve. (1 Mark)

In [49]:
```
# Remove _________ and complete the code

# Predict on train data
y_scores_svm=model.predict_proba(X_train_scaled)

precisions_svm, recalls_svm, thresholds_svm = precision_recall_curve(y_train, y_sco

# Plot values of precisions, recalls, and thresholds
plt.figure(figsize=(10,7))
plt.plot(thresholds_svm, precisions_svm[:-1], 'b--', label='precision')
plt.plot(thresholds_svm, recalls_svm[:-1], 'g--', label = 'recall')
plt.xlabel('Threshold')
plt.legend(loc='upper left')
plt.ylim([0,1])
plt.show()
```



Write your answers here: In this case the threshold value of 0.4 seems to be good as it has good recall and precision.

In [50]:
```
optimal_threshold_svm=0.4
```

Question 5.4: Check the performance of the model on train and test data using the optimal threshold. (2 Marks)

In [51]:
```
# Remove _________ and complete the code

y_pred_train_svm = model.predict_proba(X_train_scaled)
metrics_score(y_train, y_pred_train_svm[:,1]>optimal_threshold_svm)
```

```
              precision    recall  f1-score   support

           0       0.86      0.85      0.85     17073
           1       0.69      0.71      0.70      8319

    accuracy                           0.80     25392
   macro avg       0.77      0.78      0.78     25392
weighted avg       0.80      0.80      0.80     25392
```

Write your answers here: The model performance has improved by selecting the optimal threshold of 0.4. The f-1 score of the cancelled reservations has increased from 67% to 70%. It has a better recall in the cancelled reservations.

In [52]:
```
# Remove _________ and complete the code

y_pred_test = model.predict_proba(X_test_scaled)
metrics_score(y_test, y_pred_test[:,1]>optimal_threshold_svm)
```

```
              precision    recall  f1-score   support

           0       0.85      0.84      0.85      7317
           1       0.68      0.71      0.69      3566

    accuracy                           0.80     10883
   macro avg       0.77      0.77      0.77     10883
weighted avg       0.80      0.80      0.80     10883
```

Write your answers here: SVM model with linear kernel is not overfitting as the accuracy has the value of 80% for both train and test dataset. The recall of the cancelled reservations is higher compared to the above models. But the precision (cancelled reservations) is a little lower. The f-1 score of the cancelled reservations has increased from 67% to 69%.

Question 5.5: Build a Support Vector Machines model using an RBF kernel (1 Mark)

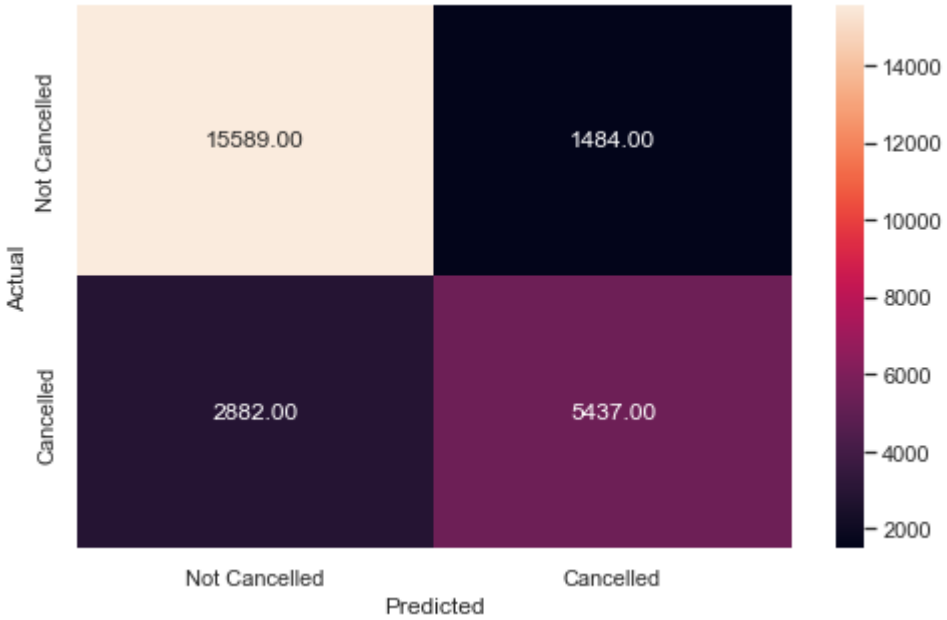
In [53]:
```
# Remove _________ and complete the code

svm_rbf=SVC(kernel='rbf',probability=True)
svm_rbf.fit(X_train_scaled,y_train)
```

Out[53]:
```
SVC
SVC(probability=True)
```

Question 5.6: Check the performance of the model on train and test data (2 Marks)

In [54]:
```
# Remove _________ and complete the code

y_pred_train_svm = svm_rbf.predict(X_train_scaled)
metrics_score(y_train, y_pred_train_svm)
```

```
              precision    recall  f1-score   support

           0       0.84      0.91      0.88     17073
           1       0.79      0.65      0.71      8319

    accuracy                           0.83     25392
   macro avg       0.81      0.78      0.80     25392
weighted avg       0.82      0.83      0.82     25392
```



Write your answers here: When compared to the SVM model with linear kernel, the model's performance on training data has been slightly improved by using an RBF kernel, considering a higher accuracy (83%) and higher f-1 scores (88% and 71% for not cancelled and cancelled reservations respectively).

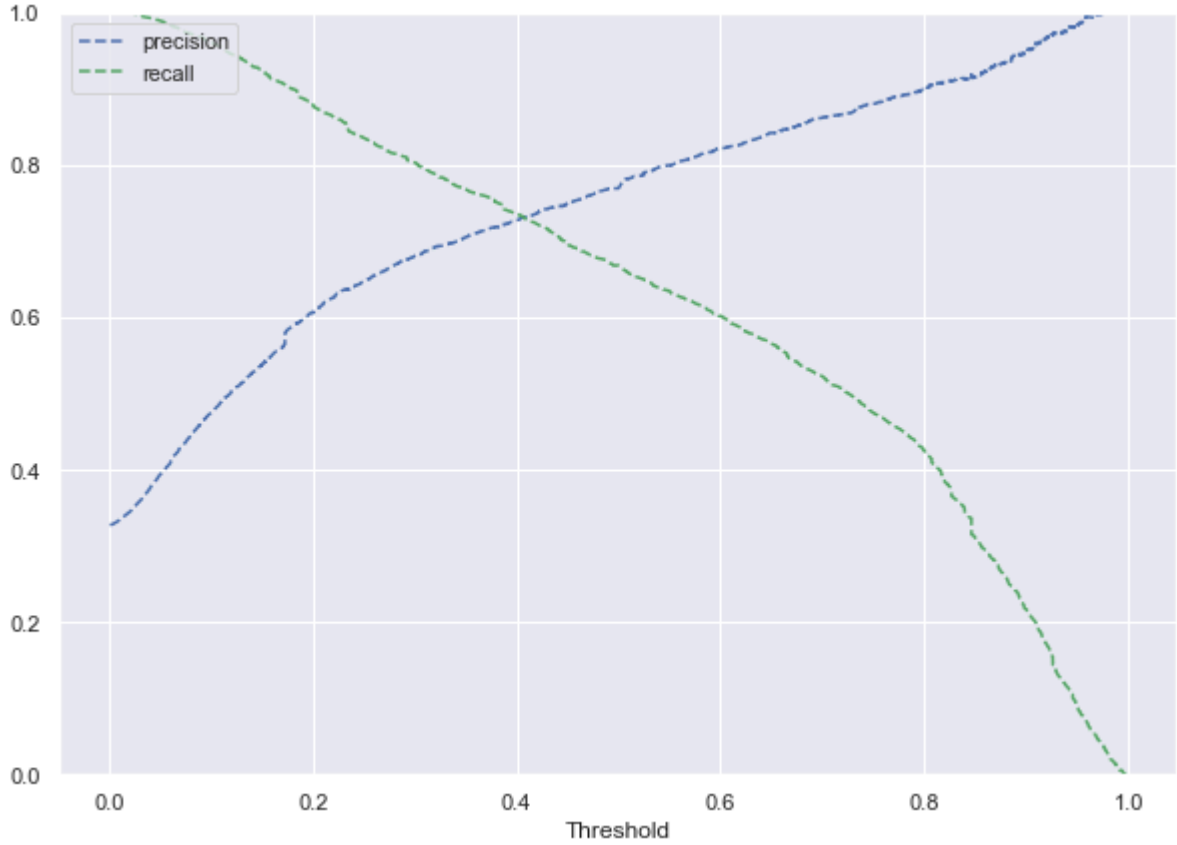
Checking model performance on test set

In [55]:
```
# Remove _________ and complete the code

y_pred_test = svm_rbf.predict(X_test_scaled)

metrics_score(y_test, y_pred_test)
```

```
              precision    recall  f1-score   support

           0       0.84      0.91      0.87      7317
           1       0.78      0.63      0.70      3566

    accuracy                           0.82     10883
   macro avg       0.81      0.77      0.78     10883
weighted avg       0.82      0.82      0.81     10883
```

Write your answers here: When compared to the baseline SVM model with linear kernel, the f-1 scores on testing data has increased from 86% to 87% for not cancelled reservations and increased from 67% to 70% for cancelled reservations.

In [56]:
```
# Predict on train data
y_scores_svm=svm_rbf.predict_proba(X_train_scaled)

precisions_svm, recalls_svm, thresholds_svm = precision_recall_curve(y_train, y_sco

# Plot values of precisions, recalls, and thresholds
plt.figure(figsize=(10,7))
plt.plot(thresholds_svm, precisions_svm[:-1], 'b--', label='precision')
plt.plot(thresholds_svm, recalls_svm[:-1], 'g--', label = 'recall')
plt.xlabel('Threshold')
plt.legend(loc='upper left')
plt.ylim([0,1])
plt.show()
```



In [57]:
```
optimal_threshold_svm=0.40
```

Question 5.7: Check the performance of the model on train and test data using the optimal threshold. (2 Marks)

In [58]:
```
# Remove _________ and complete the code

y_pred_train_svm = model.predict_proba(X_train_scaled)
metrics_score(y_train, y_pred_train_svm[:,1]>optimal_threshold_svm)
```

```
              precision    recall  f1-score   support

           0       0.86      0.85      0.85     17073
           1       0.69      0.71      0.70      8319

    accuracy                           0.80     25392
   macro avg       0.77      0.78      0.78     25392
weighted avg       0.80      0.80      0.80     25392
```

Write your answers here: SVM model with RBF kernel is performing similar to the linear kernel (the same f-1 scores). Also, is the same accuracy. The precision and recall values are the same. Maybe that's because we use the same optimal threshold.

In [59]:
```
# Remove _________ and complete the code

y_pred_test = svm_rbf.predict_proba(X_test_scaled)
metrics_score(y_test, y_pred_test[:,1]>optimal_threshold_svm)
```

```
              precision    recall  f1-score   support

           0       0.87      0.86      0.86      7317
           1       0.72      0.73      0.72      3566

    accuracy                           0.82     10883
   macro avg       0.79      0.79      0.79     10883
weighted avg       0.82      0.82      0.82     10883
```

Write your answers here: This is the best performing model when compared to SVM with linear kernel and Logistic Regression because it provides great f-1 scores (86%

and 72%). The recall value of the cancelled reservation has improved. Also, the precision of the cancelled reservations is higher compared to the above models.

Question 6: Decision Trees (7 Marks)

Question 6.1: Build a Decision Tree Model (1 Mark)

In [60]:
```
# Remove _________ and complete the code

model_dt = DecisionTreeClassifier(random_state=1)
model_dt.fit(X_train, y_train)
```

Out[60]:
```
▾ DecisionTreeClassifier
DecisionTreeClassifier(random_state=1)
```

Question 6.2: Check the performance of the model on train and test data (2 Marks)

In [61]:
```
# Remove _________ and complete the code

# Checking performance on the training dataset
pred_train_dt = model_dt.predict(X_train)
metrics_score(y_train, pred_train_dt)
```

```
              precision    recall  f1-score   support

           0       0.99      1.00      1.00     17073
           1       1.00      0.99      0.99      8319

    accuracy                           0.99     25392
   macro avg       1.00      0.99      0.99     25392
weighted avg       0.99      0.99      0.99     25392
```

Write your answers here: Very few errors on the training set, we have shockingly higher values of precision, recall and f-1 scores. The model appears to perform very well at this moment. It is important to check the performance on test data to see if the model is overfitting.

Checking model performance on test set

In [62]:
```
pred_test_dt = model_dt.predict(X_test)
metrics_score(y_test, pred_test_dt)
```

```
              precision    recall  f1-score   support

           0       0.90      0.90      0.90      7317
           1       0.79      0.79      0.79      3566

    accuracy                           0.87     10883
   macro avg       0.85      0.85      0.85     10883
weighted avg       0.87      0.87      0.87     10883
```

Write your answers here: It seems that the decision tree model has some overfitting because of the difference between the recall and precision values of the cancelled reservations obtained at test and the same values obtained at training. I recognize that the decision tree has better performance compared to Logistic Regression and SVM models. Let's see the results if we tune the decision tree.

Question 6.3: Perform hyperparameter tuning for the decision tree model using GridSearch CV (1 Mark)

Note: Please use the following hyperparameters provided for tuning the Decision Tree. In general, you can experiment with various hyperparameters to tune the decision tree, but for this project, we recommend sticking to the parameters provided.

In [63]:
```
# Remove _________ and complete the code

# Choose the type of classifier.
estimator = DecisionTreeClassifier(random_state=1)

# Grid of parameters to choose from
parameters = {
    "max_depth": np.arange(2, 7, 2),
    "max_leaf_nodes": [50, 75, 150, 250],
    "min_samples_split": [10, 30, 50, 70],
}
```

```
# Run the grid search
grid_obj = GridSearchCV(estimator, parameters, cv=5,scoring='recall',n_jobs=-1)
grid_obj = grid_obj.fit(X_train, y_train)

# Set the clf to the best combination of parameters
estimator = grid_obj.best_estimator_

# Fit the best algorithm to the data.
estimator.fit(X_train, y_train)
```

Out[63]:

```
▾ DecisionTreeClassifier
DecisionTreeClassifier(max_depth=6, max_leaf_nodes=50, min_samples_split=
10,
                       random_state=1)
```

Question 6.4: Check the performance of the model on the train and test data using the tuned model (2 Mark)

Checking performance on the training set

In [64]:

```
# Remove _________ and complete the code

# Checking performance on the training dataset
dt_tuned = estimator.predict(X_train)
metrics_score(y_train,dt_tuned)
```

```
              precision    recall  f1-score   support

           0       0.86      0.93      0.89     17073
           1       0.82      0.68      0.75      8319

    accuracy                           0.85     25392
   macro avg       0.84      0.81      0.82     25392
weighted avg       0.85      0.85      0.84     25392
```

Write your answers here: Decision tree model is working well. Tuned moded has provided a performance with unbalanced recall values (93% for not cancelled

reservations and 68% for cancelled reservations). The f-1 scores seem good and most of the precision and recall values are pretty good.

In [65]:
```
# Remove _________ and complete the code

# Checking performance on the training dataset
y_pred_tuned = estimator.predict(X_test)
metrics_score(y_test, y_pred_tuned)
```

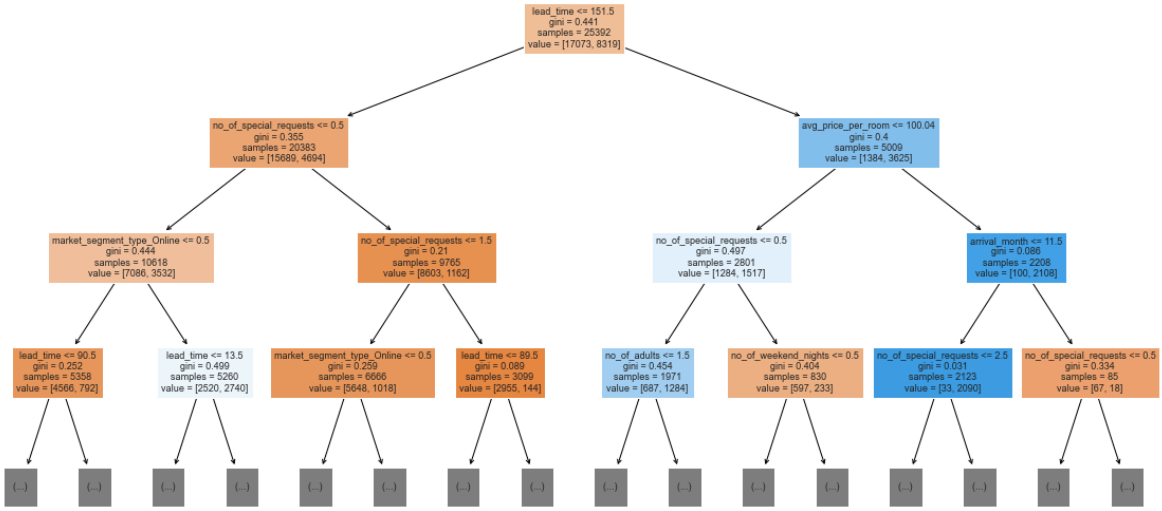
```
              precision    recall  f1-score   support

           0       0.85      0.93      0.89      7317
           1       0.82      0.67      0.74      3566

    accuracy                           0.84     10883
   macro avg       0.84      0.80      0.81     10883
weighted avg       0.84      0.84      0.84     10883
```

Write your answers here: It doesn't seem to be overfitting, it has a decent performance. Tuned moded has provided a performance with unbalanced recall values (93% for not cancelled reservations and 67% for cancelled reservations). The f-1 scores seem good, but the model performance hasn't signficantly improved.

Visualizing the Decision Tree

In [66]:
```
feature_names = list(X_train.columns)
plt.figure(figsize=(20, 10))
out = tree.plot_tree(
    estimator,max_depth=3,
    feature_names=feature_names,
    filled=True,
    fontsize=9,
    node_ids=False,
    class_names=None,
)
# below code will add arrows to the decision tree split if they are missing
for o in out:
    arrow = o.arrow_patch
    if arrow is not None:
        arrow.set_edgecolor("black")
```

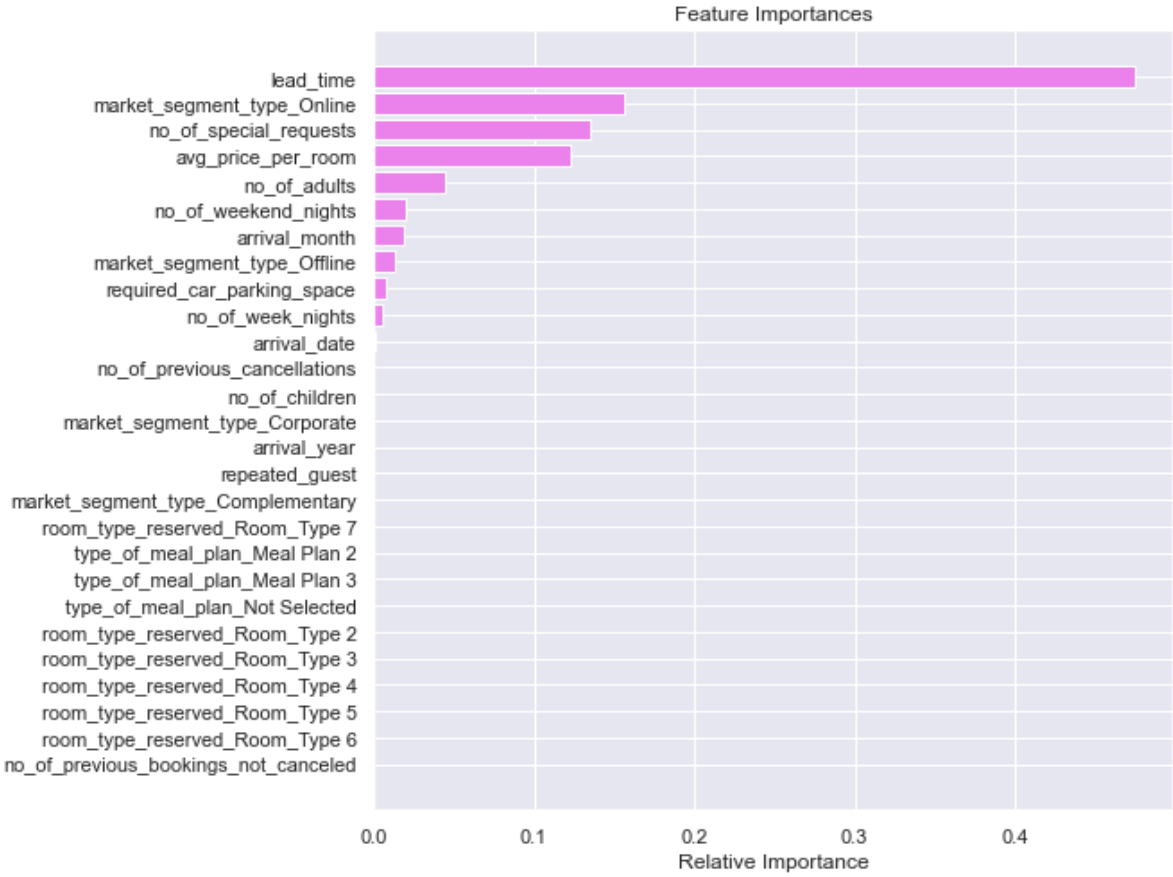
```
        arrow.set_linewidth(1)
plt.show()
```



Question 6.5: What are some important features based on the tuned decision tree? (1 Mark)

In [67]:

```
# Remove _________ and complete the code

# Importance of features in the tree building

importances = estimator.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize=(8, 8))
plt.title("Feature Importances")
plt.barh(range(len(indices)), importances[indices], color="violet", align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.show()
```



Write your answers here: The most important features are lead time, market segment type online, number of special requests and average price per room.

Question 7: Random Forest (4 Marks)

Question 7.1: Build a Random Forest Model (1 Mark)

In [68]:
```
# Remove _________ and complete the code

rf_estimator = RandomForestClassifier(random_state = 1)

rf_estimator.fit(X_train, y_train)
```

Out[68]:
```
▾ RandomForestClassifier
RandomForestClassifier(random_state=1)
```

Question 7.2: Check the performance of the model on the train and test data (2 Marks)

In [69]:
```
# Remove _________ and complete the code

y_pred_train_rf = rf_estimator.predict(X_train)

metrics_score(y_train, y_pred_train_rf)
```

```
              precision    recall  f1-score   support

           0       0.99      1.00      1.00     17073
           1       1.00      0.99      0.99      8319

    accuracy                           0.99     25392
   macro avg       0.99      0.99      0.99     25392
weighted avg       0.99      0.99      0.99     25392
```

Write your answers here: There are very few errors on the training set, the majority of samples have been classified correctly. The model has performed well on the training set.

In [70]:
```
# Remove _________ and complete the code

y_pred_test_rf = rf_estimator.predict(X_test)

metrics_score(y_test, y_pred_test_rf)
```

```
              precision    recall  f1-score   support

           0       0.91      0.95      0.93      7317
           1       0.88      0.80      0.84      3566

    accuracy                           0.90     10883
   macro avg       0.90      0.88      0.88     10883
weighted avg       0.90      0.90      0.90     10883
```

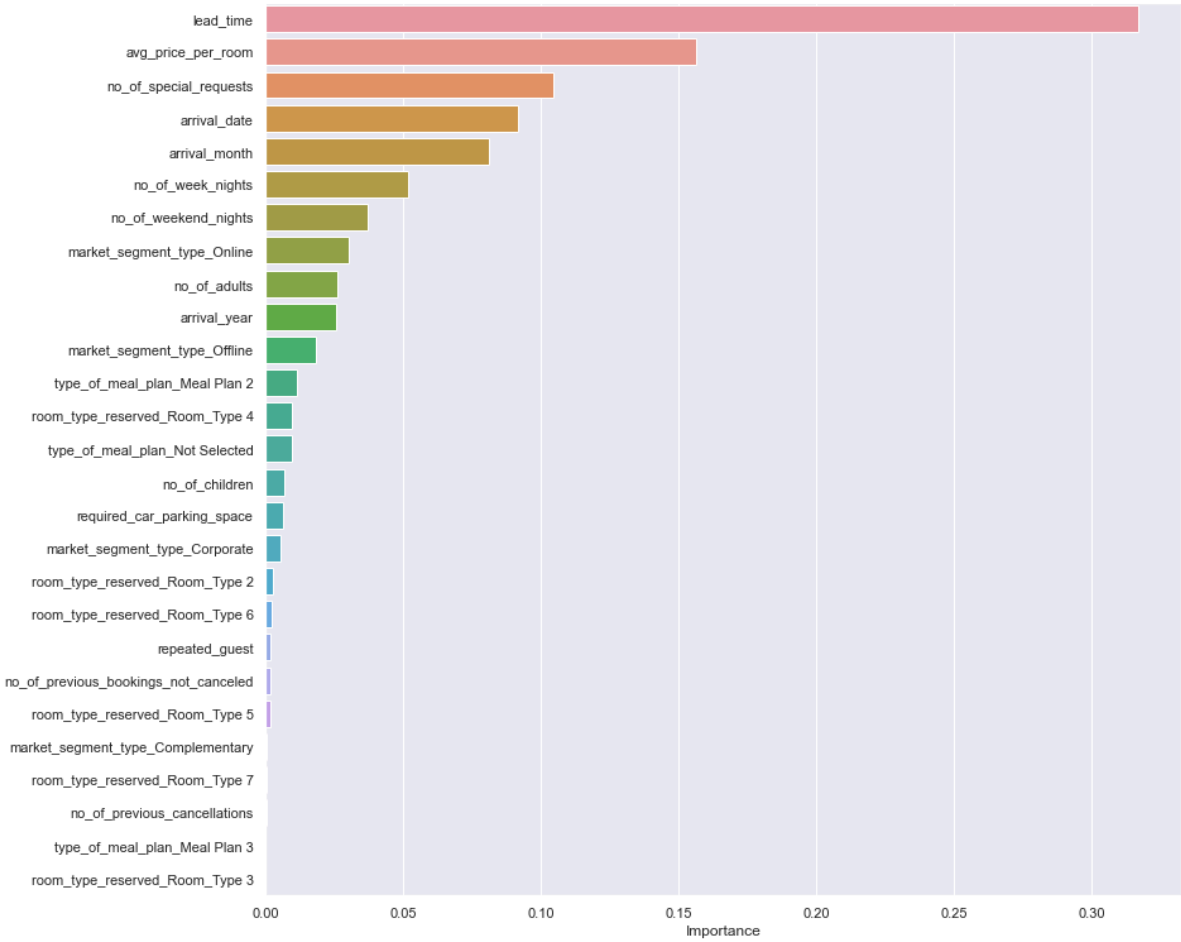
Write your answers here: The Random Forest classifier seems to be overfitting a little bit (recall values of cancelled reservations), but not so much. However, the accuracy seems to be pretty good (90%), better than the above models. Also, the f-1 scores, precision and recall values, are significantly higher.

Question 7.3: What are some important features based on the Random Forest? (1 Mark)

Let's check the feature importance of the Random Forest

In [71]:
```
# Remove _________ and complete the code

importances = rf_estimator.feature_importances_

columns = X.columns

importance_df = pd.DataFrame(importances, index = columns, columns = ['Importance']

plt.figure(figsize = (13, 13))

sns.barplot(importance_df.Importance, importance_df.index)
```

Out[71]:
```
<AxesSubplot:xlabel='Importance'>
```



Write your answers here: The Random Forest verifies some of the results from the decision tree, that the most important features are lead time, average price per room and number of special requests. Lead time is the most important feature. If the lead time of the guest is high he/she is most likely to cancel the reservation. The average price per room is also a key feature, probably because guests are looking for an affordable accommodation, so they change of opinion if they found a cheaper option.

Question 8: Conclude ANY FOUR key takeaways for business recommendations (4 Marks)

Write your answers here: -Our analysis shows that a lot of guests with higher lead times are prone to cancel, so the company should formulate a policy that establishes the penalization and refund depending on the difference between the lead time and the cancellation. -Also, it is important to consider the time between the cancellation and the expected arrival date to establish a policy that penalizes depending on the difference. If the date of cancellation is far from the original arrival date, the company can consider a refund. -In addition to that, the company could implement a restriction for the lead time, not allowing a lot of time between the reservation and the arrival date, considering the average price per room. For example, they can restrict the number of changes. If the average price per room is very cheap and there is a lot of lead time, maybe the company could restrict even more the number of changes. -When implementing a penalizing/refunding strategy, it would be great to take into account the average price per room. If the price per room is expensive, the company can consider an attractive refund if the time between the cancellation and the expected arrival date is long enough.

Happy Learning!